PROGRESS REPORT

Bill Scanning Reward & Intelligence System

An OCR + ML pipeline for receipt scanning, fraud detection and intelligent reward allocation

Team ARAJ - Ashfaaq | Arpan | Jyoti | Ranjeet

Deliverable for Prof. Uma Sankara Rao - Revision 4 - Report Date: 20 August 2026

1,973	0.942	0.864	13.2%
receipt corpus	category macro-F1	fraud AUC, real receipts	anomaly FPR

Revision 4 covers the close of the ML sprint: all five models now trained and wired, the collaborative filter and reward evaluation delivered, and the whole pipeline verified against live Firestore and live Gemini — a run which exposed twelve integration defects that component tests had not, including two models this report previously described as live that had never executed on a real upload.

1. Introduction & Methodology

The objective of this project is to build an end-to-end pipeline that scans receipts, extracts structured data using OCR, and detects fraudulent manipulations before reward points are issued. The dataset comprises three primary sources, consolidated into a single training corpus.

CORD
Public dataset - annotated receipt images

SROIE
Public dataset - scanned receipt images

Indian Receipts
100 manual photos - shops, food, pharmacy

Data Provenance Map

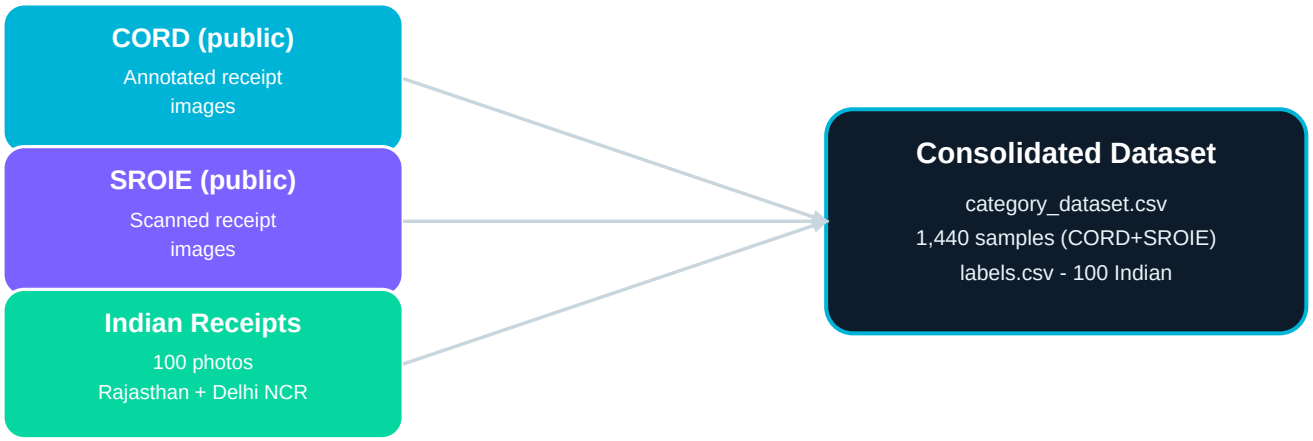

Figure 1: Sources consolidated into the training corpus. Indian receipts collected across Rajasthan and Delhi NCR.

Labelling Process (Jyoti)

The 100 Indian receipts were manually labelled into four categories:

genuine
Clean, valid receipt, no tampering

tampered
Edits, bad totals, duplicated content

multi_bill
One image with multiple bills

handwritten_edit
Handwritten edits on printed bill

Tampered Generation (Ranjeet): Using `generate_tampered.py`, additional tampered versions (v1 and v2) of the Indian receipts were created via brightness adjustment, number overwriting and duplicate copying - enriching the fraud-detection training set.

System Pipeline: Bill → Reward

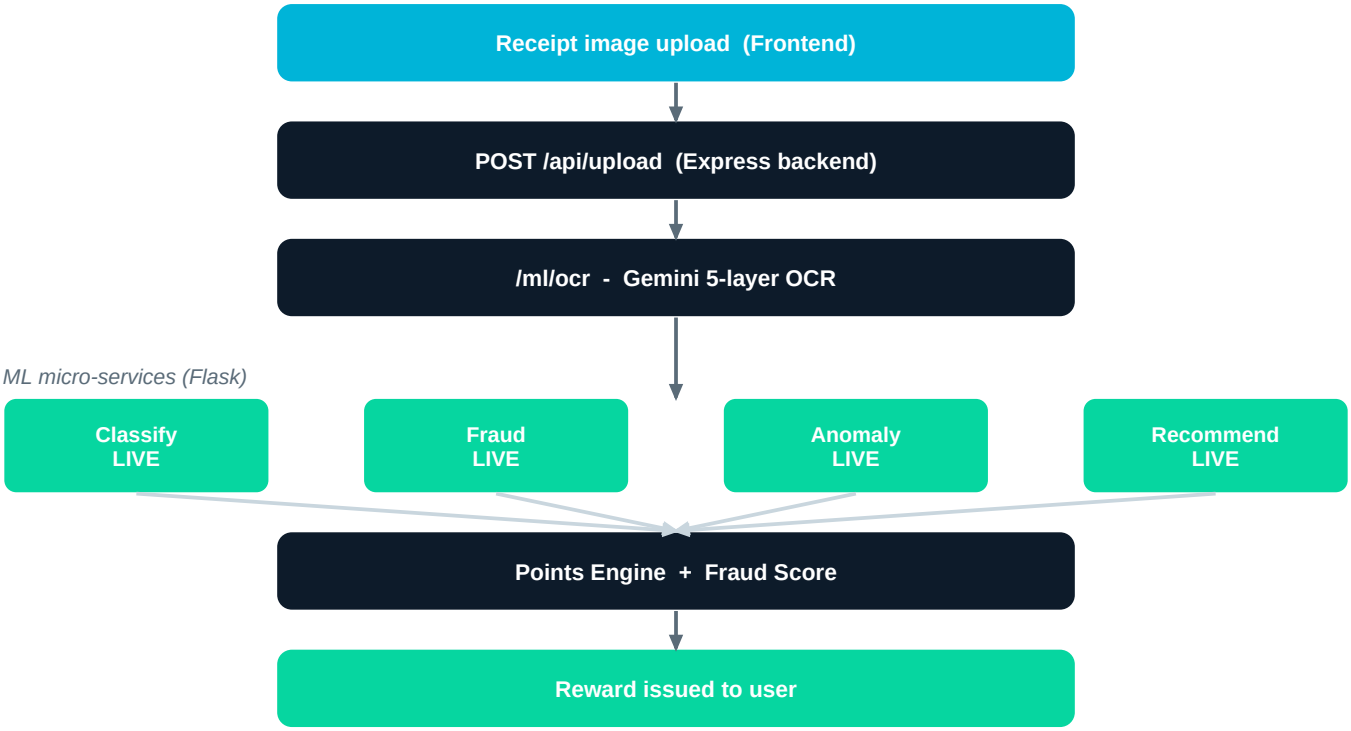

Figure 2: End-to-end flow. Every stage is live: all five ML components are trained and serving on the upload path. The fraud stage combines OCR rule signals, a perceptual-hash duplicate check and the 448px tamper CNN.

2. Dataset Statistics & Exploratory Analysis

2.1 Overall Dataset Composition

The consolidated corpus (`receipts_master.csv`) holds **1,973 receipts** — 800 CORD, 973 SROIE and 200 team-labelled Indian receipts. After dropping rows with empty text, **1,427** are usable for spend-category training. The target changed since v2 of this report. The earlier two-class *Restaurant vs Retail* label was found to be **1:1 with the corpus source** — a model could score well by identifying which dataset a receipt came from rather than what was bought. That is leakage, so the target is now a three-class spend category and `source` is excluded from the features.

Spend category (category_3class)	Count	Share
General Retail	955	66.9%
Food & Beverage	397	27.8%
Supermarket / Grocery	75	5.3%
Total	**1,427**	**100%**

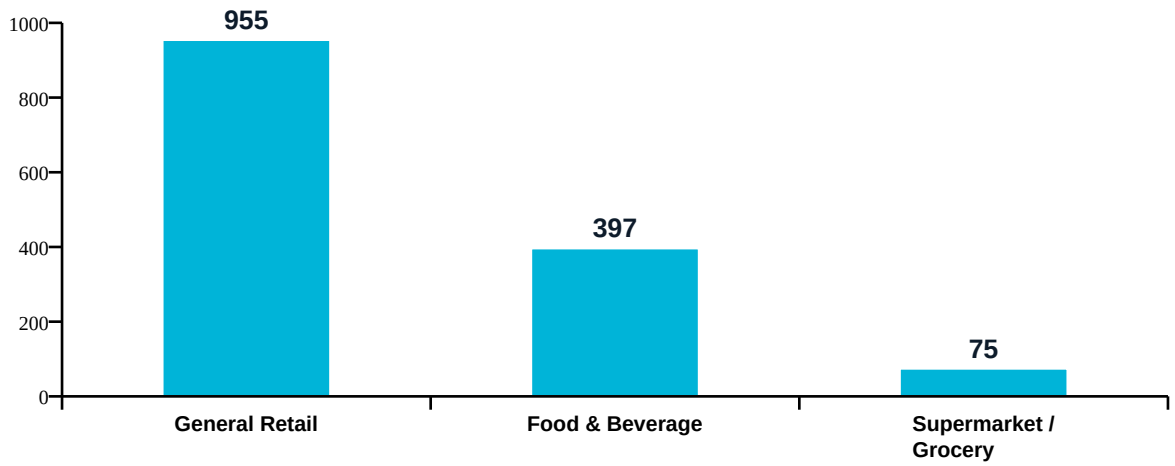

Figure 3: Three-class spend distribution. Grocery is the minority class at 75 samples — the reason macro-F1, not accuracy, is the selection metric.

2.2 Null Value Analysis

A thorough null check revealed missing values across metadata columns. This is structural rather than random: CORD contributes 800 rows that never recorded a merchant or date, while SROIE records both for every row. Imputing them would be inventing data, so date and merchant are **not used as features** — the classifier works from item text, which is present throughout.

Column	Missing Count
merchant	454
address	455
date	608
total	3
items_text	973

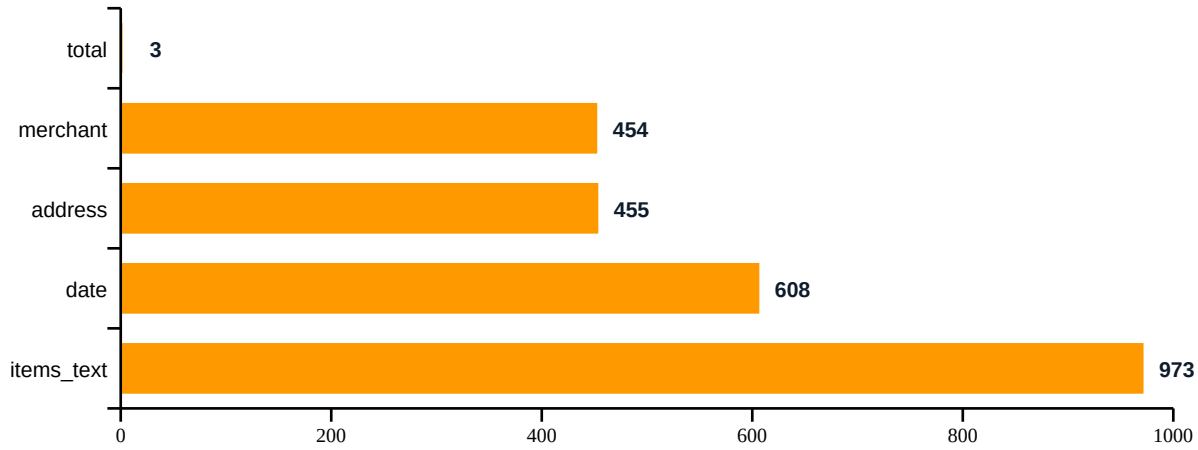

Figure 4: Missing values by column (out of 1,427 rows).

2.3 Fraud Label Distribution (200 labelled receipts)

Correction to v2 of this report. The previous revision reported this split with genuine and tampered **transposed**. The correct figures are below: **tampered is the majority class and genuine is the minority**, which is the opposite of the usual fraud setting and materially affects how the model is trained and read.

Label	Count	Share
Tampered	129	64.5%
Genuine	65	32.5%
Multi-bill	5	2.5%
Handwritten	1	0.5%
Total	**200**	**100%**

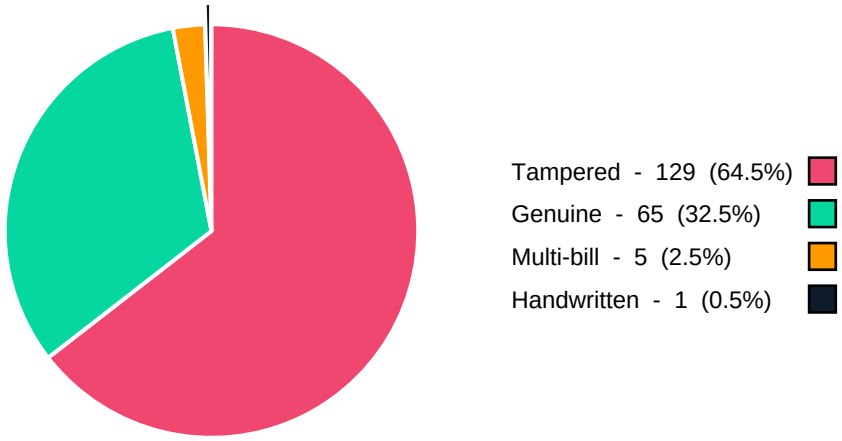

Figure 5: Fraud-label distribution across the 200 labelled receipts. Only 65 genuine images exist, and that ceiling is the single largest constraint on the fraud model.

194 of the 200 are usable for binary classification, and they come from only **103 source receipts** — 22 appear as both a genuine and a tampered version. Splits are therefore grouped on the source receipt (Section 5.1).

3. Fraud Detection Signals & OCR Challenges

Fraud detection is not limited to manual labels. Automated challenge handlers flag suspicious receipts; these signals aggregate into a final fraud score per upload.

#	Signal	Technique
1	Blur detection	OpenCV Laplacian variance; rejects images below threshold (100).
2	Multi-bill anomaly	Flags abnormally high text-region counts (merged documents).
3	Duplicate check	Perceptual hash (pHash) + merchant-date-total matching.
4	Handwritten edits	Gemini confidence + pixel-density analysis of overwrites.
5	Rate-limit handling	Exponential backoff for external API calls (Gemini).

4. Model 1 — Spend-Category Classifier

TF-IDF over item text and merchant, then three algorithms compared under identical features and splits. The winner was chosen on **validation** macro-F1; the test set was touched once, afterwards. That ordering is what makes the test figure an honest estimate rather than an optimistic one.

Model	Val macro-F1	Test macro-F1	Outcome
Random Forest	**0.909**	**0.942**	**Selected**
Linear SVM	0.891	—	Rejected
Logistic Regression	0.890	—	Rejected

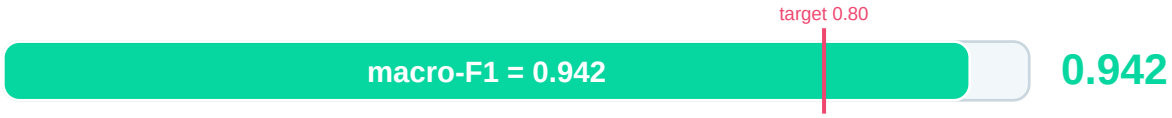

Figure 6: Test macro-F1 0.942 against a 0.80 target (accuracy 0.944).

Read with one caveat. The test score is *higher* than validation (0.942 vs 0.909), which is the wrong direction. It comes almost entirely from the Grocery class moving from F1 0.83 to 0.96 — on only 11 test samples, where one or two receipts shift the figure by ~0.08. The honest headline is **approximately 0.94 with a small-class caveat**. The genuine weak spot is Food & Beverage recall at 0.87.

5. Model 2 — Fraud Tamper Detection

This is where most of the sprint went, and where the most important work was not training a model but auditing how it was being measured.

5.1 Three evaluation faults found and corrected

The Sprint 2 figure could not be reproduced. Investigating why surfaced three separate faults, each of which had been inflating the result. **These are distinct from the twelve integration defects noted in the conclusion:** the three here concern how the model was *measured*, and inflated a reported number; the twelve concern whether the code actually *ran*, and left components that had been evaluated correctly doing nothing on a live upload.

#	Fault	Effect	Correction
1	23 PDF receipts unopenable by the imaging library; the loader only checked that the path existed	19 were genuine — about a third of the minority class was silently missing from training	Rasterised to images
2	Source-receipt leakage: 194 images come from 103 receipts, 22 appearing as both genuine and tampered	A random split put the same receipt in train and test — the model could recognise the receipt, not the tampering	Splits grouped on source receipt
3	Tampered images had been re-encoded by the editing script	A classifier scored **AUC 0.690 from the JPEG quantisation table alone**, with no image content whatsoever	Every image re-encoded identically

5.2 Input resolution — the largest single gain

Receipts are roughly 1200×1600 pixels, but the model was seeing 224×224. That is a 7.1× downscale, which reduces an overwritten digit to about six pixels — there is almost nothing left to detect. Retraining at 448 preserves roughly twelve pixels of the same evidence.

Evaluation set	224×224	448×448	Change
All 194 images (pooled out-of-fold)	0.752	**0.805**	+0.053
94 real receipts (source-confound control)	0.781	**0.864**	+0.083

Confirmed, not asserted. A paired bootstrap clustered on the source receipt gives +0.053, CI [+0.008, +0.100], p = 0.020 overall, and +0.083, CI [+0.015, +0.161], p = 0.016 on real receipts.

0.864 is the figure we stand behind. It is measured on the 94 real photographed receipts, where genuine and tampered images share a camera, so the model cannot be detecting image provenance instead of tampering. The pooled 0.805 includes 100 synthetically tampered images. Both are **below the 0.90 target**, and Section 7 explains why we believe that is a data limit rather than a model one.

5.3 A negative result, reported

Combining the CNN with 31 hand-designed forensic features made results **worse**, not better: 0.790 pooled against 0.805 for the CNN alone, and 0.833 against 0.864 on real receipts. A learned stacker was worse still at 0.758. The two signals correlate at Spearman 0.638, so the forensic features add mostly redundancy plus their own noise. The CNN alone is what is served.

6. Models 3, 4 and 5 — Anomaly, Recommendation and Reward Evaluation

6.1 Spending-anomaly detector (unsupervised)

An Isolation Forest over the log-ratio of a transaction amount to a reference amount — the population median for that category, replaced by the user's own median once they reach five receipts. Three algorithms were compared under identical features, splits and outlier budget, with the selection rule fixed in advance.

Model	FPR (real, held-out)	Recall @10×	Outcome
Isolation Forest	**13.2%**	**100%**	**Selected**
One-Class SVM	11.8%	0%	Rejected
Local Outlier Factor	13.6%	0%	Rejected

Two honesty notes. First, the 13.2% false-positive rate is **set** by the contamination parameter, not discovered — it clears the 15% target by construction, and the earned number beside it is the recall column. Second, One-Class SVM was rejected *despite the lowest FPR* because its recall profile is non-monotonic (26% at 5×, 0% at 10×, 100% at 20×), which is unusable for review. Recall is 100% at 10× and above but **0% at 3× and 5×**: the detector cannot catch modest inflation.

A second negative result: adding category and day-of-week features made it worse, cutting recall at 10× from 100% to 39%. The served model is deliberately the simpler one.

6.2 Recommender

Notebook 04 landed on 19 August, so ranking is no longer content-only. Offers are scored against a decayed per-category interest vector built from the user's own history, blended 50/50 with an SVD collaborative estimate; the service reports `model: collaborative+content`. Notebook 05 followed as an end-to-end evaluation harness.

Metric	Measured	Target	Status
SVD 5-fold CV RMSE	**0.9157**	<0.5 (notebook) / <1.0 (plan)	**Not claimed**
NDCG@5	**0.7984**	> 0.70	**Cleared, not claimed**
Precision@5 / Recall@5	0.50 / 0.8833	—	Recall corrected from 2.05

Neither target is claimed, and the NDCG one clears its bar. Both the factorisation and its evaluation run on the same generated file — 772 interactions across 60 synthetic users, where the rating is a deterministic function of a preference the generator itself assigned, and the relevance labels used to score the ranking derive from that same rule. A model that recovers the generator scores well by construction, so the metric cannot meaningfully fail. The live Firestore export holds 19 transactions across three accounts, far too sparse to replace it.

Two further corrections before these figures were reported. Recall@5 was originally 2.05 — impossible, since recall is bounded at 1.0; the numerator counted recommended offers (up to five) against a denominator of relevant categories (at most three). And the RMSE target exists in two places with two different values, 0.5 in the notebook header and 1.0 in the sprint plan, so 0.9157 is a pass or a fail depending which document is consulted. Both are recorded rather than resolved silently. Notebook 05 was also delivered as an evaluation harness rather than the XGBoost ranker the plan specified, so its ‘three algorithms compared’ criterion is **unmet**.

7. Next Steps & Conclusion

On the 0.90 fraud target. Two unrelated methods — a convolutional network and 31 hand-designed forensic features — plateau at a similar level, and fusing them does not help. Independent methods hitting the same ceiling is evidence about the corpus rather than the model. With 65 genuine receipts drawn from 103 source images, the route to 0.90 is 500–1,000 more genuine receipts, not a different architecture.

Code is frozen except for Notebooks 04 and 05. Remaining work:

#	Action	Owner	Status
1	Notebooks 04 (collaborative filter) and 05 (reward evaluation)	Arpan	**Closed** 19 Aug
2	Merge dev → main and tag the v1.0-fyp release	Ashfaaq	**Closed** 20 Aug
3	Pipeline verified against live Firestore and live Gemini	Ashfaaq	**Closed** 20 Aug
4	TC-26 — four demo scenarios through the browser, with screenshots	Ashfaaq	Open
5	Deploy backend and ML service (Render) + frontend (Vercel)	Ashfaaq	Open
6	Capture the four UI screenshots and record the demo video	Ashfaaq	Open
7	Two-page summary for Prof. Uma	Jyoti	Open
8	Mock viva and per-member rehearsal	Team	Open

Conclusion. Five models are trained, compared against alternatives and wired into the live upload path; two cleared their targets with margin, one is short of a demanding target for a reason the evidence identifies, and two are measured but deliberately not claimed. **The most valuable outcome of this revision is not a number at all.** Running the whole stack end to end for the first time revealed that two components this report had described as live — the tamper CNN and the perceptual-hash duplicate check — had never executed on a real upload: one was called without the image, the other was never given anything to compare against. Every unit test passed throughout. Twelve such defects were found and fixed across three sessions, and the lesson is recorded rather than quietly absorbed: a component test cannot see a seam. The evaluation figures in this report were measured directly against the image corpus and are unaffected, but the deployed system did not behave as described until those fixes landed. Three negative results are reported rather than omitted. Every figure here is produced by a committed script and can be regenerated on request.

Team ARAJ

Ashfaaq · Arpan · Jyoti · Ranjeet

BITS Pilani Digital - Group 120